

作业 11

——线性时间 $(1 + \epsilon)$ 近似直径——

声明：题目来源于 <http://openjudge.cn/>，解题思路参考教授上课讲解内容，代码实现由本人完成。

分享这个文档仅限于学术交流，不用于任何形式的商业用途。

1. 题目

描述

在二维平面上有一个包含 n 个点的点集，你需要估计这个点集的直径，即最远点对的距离。

你需要保证你的输出与正确答案的相对误差不超过 10%，即设正确答案为 ans ，你的输出为 out 需要满足 $|ans - out| / ans \leq 0.1$ 。

输入

输入数据的第一行为一个正整数 $n \leq 10^5$ ，表示点集大小。接下来 n 行，每行两个绝对值不超过 10^9 的实数（小数点后至多有6位），表示一个点的坐标。

输出

输出一个实数，表示直径的大小。

样例输入

```
10
5.078947 4.887441
8.526691 1.958934
9.126218 4.857025
-3.691721 7.462236
-8.573120 3.106406
0.104984 6.977667
-3.588661 7.607419
8.785334 2.726462
5.217877 4.397641
0.087251 -1.744233
```

样例输出

```
17.785703
```

2. 思路

显然，如果我们把所有点两两之间的距离算出来是非常费劲的事情，计算机也会累的。秉持着能躺着绝不坐着的原则，我们要想办法偷点懒，节约点电费来个自己买咖啡。

于是，聪明的你想到了一个办法，我们不用考虑所有的点的细节，我们考察一些在一个区域内的点，然后选取这个区域的中心代表所有点就可以了。这种选区代表的思路在生活的方方面面都很常见。因此，关键在于，我们如何选择选择一个区域，如何选择代表？

地缘关系显然是非常重要的考虑因素，也就是我们希望把靠得近的点放在一个区域里面，八竿子打不着的点不能放在一个区域。

假设我们现在已经分好区域了，那么我们就将所有点都放到这个区域的中心。注意，这个问题里面，我们并不关心这个区域中的点的数目是多还是少，只要它在那里，那么就必须算上。因此，我们只需把所有有点的区域平等地看待，而不能按照人多势众的方式按照数目加权平均。

3. 算法

3.1. 先把准备工作做了吧

首先你需要把咖啡豆磨成粉末，设定自己想喝多少克，对吧。

```
int main(){
    int n;
    cin >> n;
    vector<Point> pts(n);
    for(int i = 0; i < n; ++i)
        scanf("%lf %lf", &pts[i].x, &pts[i].y);
}
```

传说 scanf 的效率比 cin 高，所以我们就用 scanf 来读入数据了。然后我们需要估算直径，也就是很随意的一件事，捞一个点当中心，然后算一算：

```
srand(time(0));
int _center = rand() % n; // 随手捞一个点作为中心
double rMax = 0.0; // 半径肯定比这个小
for(int i = 0; i < n; i++){
    double dx = pts[i].x - pts[_center].x;
    double dy = pts[i].y - pts[_center].y;
    double dist = dx * dx + dy * dy;
    if(dist > rMax) rMax = dist;
}
rMax = sqrt(rMax);
```

3.2. 如何划分网格

我么需要大体知道这些点是怎么分布的，于是，我们可以粗略地选取一个点作为“中心”，然后计算所有点与这个中心的距离，选取一个合适的距离作为网格的边长。当然，方格划分越细碎，得到的结果就越精确，但是消耗的能量也就越大，留给我的咖啡也就越少，因此，我们只需大概能满足题目的精度就可以。

```
double epsilon = 0.1;
double l = epsilon * rMax / 2.0; // 格子边长
vector<Point> centers(n); // 随手把点放回格子中心
for(int i = 0; i < n; ++i){
    double x = pts[i].x;
    double y = pts[i].y;
    double cx = floor(x / l) * l + l / 2.0;
    double cy = floor(y / l) * l + l / 2.0;
    centers[i] = {cx, cy};
}
```

3.3. 去重

正如之前说的，我们是雨露均沾，不是人多力量大，所以我们需要去重，去掉重复的中心点。稍微解释一下，这里没有直接写==来判断相等是因为 double 类型判断绝对的相等可能会出问题。erase 函数的用法我得积累一下，豆老师教我的。

```
bool cmp(const Point &a, const Point &b){
    if(fabs(a.x - b.x) < 1e-12) return a.y < b.y;
    return a.x < b.x;
}
```

```

bool eq(const Point &a, const Point &b){
    return fabs(a.x - b.x) < 1e-12 && fabs(a.y - b.y) < 1e-12;
}

sort(centers.begin(), centers.end(), cmp);
centers.erase(unique(centers.begin(), centers.end(), eq), centers.end());

```

3.4. 暴力算直径

该吃的苦还是得吃，这一步是跑不掉的。

```

double diam = 0.0;
int m = centers.size();
for(int i = 0; i < m; ++i){
    for(int j = i + 1; j < m; ++j){
        double dx = centers[i].x - centers[j].x;
        double dy = centers[i].y - centers[j].y;
        double dist = sqrt(dx * dx + dy * dy);
        if(dist > diam) diam = dist;
    }
}

```

4. 完整代码

```

#include<bits/stdc++.h>
using namespace std;
struct Point {
    double x, y;
};

bool cmp(const Point &a, const Point &b){
    if(fabs(a.x - b.x) < 1e-12) return a.y < b.y;
    return a.x < b.x;
}

bool eq(const Point &a, const Point &b){
    return fabs(a.x - b.x) < 1e-12 && fabs(a.y - b.y) < 1e-12;
}

int main(){
    int n;
    cin >> n;
    vector<Point> pts(n);
    for(int i = 0; i < n; ++i)
        scanf("%lf %lf", &pts[i].x, &pts[i].y);

    srand(time(0));
    int _center = rand() % n; // 随手捞一个点作为中心
    double rMax = 0.0; // 半径肯定比这个小
    for(int i = 0; i < n; i++){
        double dx = pts[i].x - pts[_center].x;
        double dy = pts[i].y - pts[_center].y;
        double dist = dx * dx + dy * dy;
        if(dist > rMax) rMax = dist;
    }
    rMax = sqrt(rMax);

    double epsilon = 0.1;

```

```
double l = epsilon * rMax / 2.0; //格子边长

vector<Point> centers(n);
for(int i = 0; i < n; ++i){
    double x = pts[i].x;
    double y = pts[i].y;
    double cx = floor(x / l) * l + l / 2.0;
    double cy = floor(y / l) * l + l / 2.0;
    centers[i] = {cx, cy};
}

sort(centers.begin(), centers.end(), cmp);
centers.erase(unique(centers.begin(), centers.end(), eq), centers.end());

//暴力求直径
double diam = 0.0;
int m = centers.size();
for(int i = 0; i < m; ++i){
    for(int j = i + 1; j < m; ++j){
        double dx = centers[i].x - centers[j].x;
        double dy = centers[i].y - centers[j].y;
        double dist = sqrt(dx * dx + dy * dy);
        if(dist > diam) diam = dist;
    }
}

printf("%.6lf\n", diam);
return 0;
}
```

终于写完啦！开心！
但，还得去写高数作业。。。